

基于 OpenMP 的三种排序算法并行化实现与性能分析

李佩优 PB23000270

摘要

本文针对三种经典排序算法——选择排序 (Selection Sort)、快速排序 (QuickSort) 和归并排序 (MergeSort)，实现了其 C++ 串行版本与基于 OpenMP 的并行版本，并在 10^6 至 10^7 规模的单精度浮点数数据集上进行了系统的性能基准测试。并行策略上，快速排序和归并排序采用 OpenMP Task 机制实现递归任务的并行化，并引入阈值控制任务粒度以避免过细划分带来的开销；选择排序则在每轮最小值查找中使用 `parallel for` 并行化查找过程。实验考察了串行版本与 1/2/4/8/16 线程下的并行性能，计算了加速比并分析了各算法的并行适应性。结果表明：(1) 归并排序的并行加速效果最优，在 10^7 规模下达到 $5.2\times$ 加速；(2) 快速排序次之，达到 $3.2\times$ 加速，主要受限于划分阶段的负载不均衡；(3) 选择排序因通信开销远超并行收益，始终慢于串行版本。本文验证了 OpenMP Task 机制在分治类排序算法中的有效性，并通过实验数据诠释了 Amdahl 定律在并行排序中的应用。

1 引言

排序是计算机科学中最基础且应用最广泛的问题之一。随着多核处理器的普及，利用并行计算加速排序算法成为提升性能的关键途径。OpenMP 作为共享内存并行编程的事实标准，通过编译器指令实现了串行代码的渐进式并行化，特别适合对已有串行排序算法进行改造。

本实验选取三种具有代表性的排序算法——选择排序 ($O(n^2)$)、快速排序 (期望 $O(n \log n)$) 和归并排序 ($O(n \log n)$)，旨在探索不同算法特性下并行化的效果差异。选择排序代表了数据依赖性强、并行化困难的一类算法；快速排序代表了递归分治但划分存在负载不均衡的一类算法；归并排序代表了分治结构规整、天然适合并行的一类算法。

实验依据课程要求，在 10^6 至 10^7 规模的浮点数数据集上，对每种算法的串行版本和 OpenMP 并行版本 (线程数 1/2/4/8/16) 进行了全面的性能测试，通过加速比量化并行化效果，并从算法结构的角度解释各算法并行表现的差异。

2 算法设计与实现

2.1 串行排序算法

三种串行排序算法均采用标准的教科书实现 [1]，使用 `std::vector<float>` 存储数据。

2.1.1 选择排序

选择排序在每轮迭代中从未排序部分找出最小元素，将其交换到已排序部分的末尾。时间复杂度 $O(n^2)$ ，空间复杂度 $O(1)$ 。

2.1.2 快速排序

采用 Lomuto 划分方案，每次选取中位数三（median-of-three）作为枢轴以降低最坏情况概率。递归地对左右子数组进行排序。时间复杂度期望 $O(n \log n)$ 。

2.1.3 归并排序

采用自顶向下的递归归并排序，将数组平分为两半，递归排序后再合并。合并过程中使用与数组等长的临时数组以避免频繁内存分配。时间复杂度 $O(n \log n)$ ，空间复杂度 $O(n)$ 。

2.2 OpenMP 并行排序算法

并行化的核心思路是利用 OpenMP Task 机制，将递归调用转化为可被不同线程异步执行的任务。三种算法的并行策略各有不同。

2.2.1 并行快速排序：OpenMP Task 划分

快速排序的并行化最为直接：划分完成后，左右两个子数组的递归排序互不依赖，可以并行执行。实现方式如下：

```
1 void quicksort_omp_helper(vector<float>& arr, int left, int right) {
2     if (right - left < CUTOFF) {
3         insertion_sort(arr, left, right);
4         return;
5     }
6     int p = partition(arr, left, right);
7
8     #pragma omp task shared(arr) firstprivate(left, right, p)
9     quicksort_omp_helper(arr, left, p - 1);
10    #pragma omp task shared(arr) firstprivate(left, right, p)
11    quicksort_omp_helper(arr, p + 1, right);
12    #pragma omp taskwait
13 }
```

入口处通过 `#pragma omp parallel { #pragma omp single }` 确保初始任务只由一个线程创建，随后通过 `task` 指令将子任务分配给空闲线程。`taskwait` 保证两个子任务完成后再继续执行上层代码。实验中将阈值设为 2000——当子数组长度小于此值时切换为插入排序，避免 Task 创建和调度开销超过计算收益。

2.2.2 并行归并排序：稳定的分治并行

归并排序的并行结构与快速排序类似，左右半区独立递归排序：

```
1 void mergesort_omp_helper(vector<float>& arr,
2                             int left, int right, vector<float>& temp) {
3     if (right - left < CUTOFF) {
4         mergesort_serial(arr, left, right, temp);
5         return;
6     }
7     int mid = left + (right - left) / 2;
8 }
```

```

9      #pragma omp task shared(arr, temp) firstprivate(left, mid, right)
10     mergesort_omp_helper(arr, left, mid, temp);
11     #pragma omp task shared(arr, temp) firstprivate(left, mid, right)
12     mergesort_omp_helper(arr, mid + 1, right, temp);
13     #pragma omp taskwait
14
15     merge(arr, left, mid, right, temp);
16 }

```

与快速排序的关键区别在于：归并排序的合并阶段是串行的，且每次合并需要遍历整个子数组进行复制操作。需要特别说明的是，虽然多个任务共享 `temp` 数组，但每次 `merge` 操作仅访问其对应区间 `[left, right]`，不同任务间不存在重叠写入，因此不会产生数据竞争。实验中将阈值设为 100000，在保证足够并行粒度的同时避免任务数量爆炸。

2.2.3 并行选择排序：细粒度并行

选择排序的可并行点在于每轮查找最小值的内层循环。实现将其改造为 `parallel for` 并行归约——每个线程先找出局部最小值，再通过 `critical` 区域汇总全局最小值。

由于每轮并行查找后需同步更新数组，且 n 轮外循环无法并行化，该实现的并行开销远大于计算收益。

2.3 计时与正确性验证

采用 `std::chrono::steady_clock` 进行高精度计时。每轮测试对同一数据副本排序后，使用 `std::is_sorted` 验证正确性。此外，在正式测试前使用 1000 元素的小数组，与 `std::sort` 的结果逐元素比对，确保 6 种排序函数（3 串行 + 3 并行）均正确实现。

3 实验设置

3.1 实验环境

- CPU: Intel Core i7-12700H (14 核/20 线程, P-core 4.7GHz)
- 内存: 16GB DDR5
- 编译器: MinGW-w64 GCC 8.1.0 (64-bit), 编译选项 `-std=c++17 -O2 -fopenmp`
- OpenMP 版本: 4.5

关于线程配置的说明：虽然 CPU 具备 20 线程能力，但考虑到 OpenMP 任务调度开销及实验时间成本，线程数设置为 1/2/4/8/16，已覆盖从串行到接近硬件上限的主要扩展趋势，足以展示各算法的可扩展性特征。

3.2 数据生成

使用 `std::mt19937` (Mersenne Twister) 伪随机数生成器，配合 `std::uniform_real_distribution<float>` 生成 $[-10^6, 10^6]$ 范围内的随机浮点数。每种规模使用独立种子以保证数据集各不相同。

3.3 测试方案

- 快速排序与归并排序：10 组数据集，规模依次为 $1 \times 10^6, 2 \times 10^6, \dots, 10 \times 10^6$ ，全部满足“ $\geq 10^6$ ”的要求。
- 选择排序：由于其时间复杂度为 $O(n^2)$ ，对于 10^6 规模的数据，理论比较次数约为 5×10^{11} 量级，在实验平台上运行时间将超过数小时，不具备实际测试意义。因此仅使用较小规模（ $10^3 \sim 5 \times 10^4$ ）验证其并行化效果，而快速排序和归并排序承担大规模性能测试任务。
- 线程配置：串行基准 + 并行版在 1/2/4/8/16 线程下的表现。
- 性能指标：运行时间（毫秒）、加速比（ $S = T_{\text{串行}}/T_{\text{并行}}$ ）。

4 实验结果与分析

4.1 选择排序结果

表 1: 选择排序性能数据

规模	串行 (ms)	OMP1	OMP2	OMP4	OMP8	OMP16
1 000	3.9	12.8	67.2	95.3	137.7	224.9
5 000	96.6	154.2	419.9	526.8	673.2	1098.3
10 000	383.2	546.9	976.1	1120.0	1353.8	2242.7
50 000	9581.2	11564.5	9333.7	7924.6	8813.9	11815.7

表 2: 选择排序加速比

规模	OMP1	OMP2	OMP4	OMP8	OMP16
1 000	0.306	0.058	0.041	0.028	0.017
10 000	0.701	0.393	0.342	0.283	0.171
50 000	0.828	1.027	1.209	1.087	0.811

分析：选择排序在大多数配置下加速比均小于 1（即比串行慢），仅在 50000 规模下 OMP4 勉强达到 1.209 $\times$ 。原因在于：(1) 外层 n 轮迭代存在严格的数据依赖，无法并行化，这是串行瓶颈；(2) 内层并行查找每个最小值后需要 `critical` 区域合并结果，带来了巨额同步开销；(3) 当数组规模较小时，线程创建和调度的开销占比过高。这一结果符合预期，体现了 Amdahl 定律——串行比例决定了理论上限。

4.2 快速排序结果

表 3: 快速排序性能数据与加速比

规模	串行 (ms)	OMP1	OMP2	OMP4	OMP8	OMP16	加速比
1×10^6	216.1	604.8	351.8	235.6	124.8	103.9	$2.08\times$
3×10^6	723.0	1830.8	1276.6	905.7	385.0	220.5	$3.28\times$
5×10^6	1199.9	3145.1	2313.1	1183.7	675.9	393.5	$3.05\times$
8×10^6	2041.4	5104.4	3501.0	1908.7	1029.6	630.0	$3.24\times$
1×10^7	2544.4	6358.2	4940.7	2018.6	1247.6	796.2	$3.20\times$

分析：快速排序在 16 线程下达到约 $3.2\times$ 的加速比，但远未达到线性加速。主要原因包括：

1. **任务负载不均衡**：快速排序的划分结果取决于枢轴位置，左右子数组大小不一定相等。即使采用中位数三优化，在大规模数据上仍可能出现非均衡划分，导致部分线程提前空闲。
2. **Task 调度开销**：`cutoff = 2000` 的阈值虽已避免过细任务，但递归树浅层划分出的少量大任务仍是串行瓶颈——第一层划分后的两个大子任务无法与后续任务充分重叠。
3. **OMP1 远慢于串行**：单线程下使用 Task 机制时，任务创建、调度和 `taskwait` 同步带来了约 $2.5\times$ 的额外开销，说明 Task 机制的设计初衷是为多线程并行服务而非串行执行。

4.3 归并排序结果

表 4: 归并排序性能数据与加速比

规模	串行 (ms)	OMP1	OMP2	OMP4	OMP8	OMP16	加速比
1×10^6	286.5	295.4	153.1	114.1	94.6	74.7	$3.84\times$
3×10^6	957.1	938.1	487.5	353.2	276.8	173.3	$5.52\times$
5×10^6	1650.9	1652.2	837.6	590.6	406.3	325.0	$5.08\times$
8×10^6	2707.8	2744.4	1391.8	1003.4	655.2	530.4	$5.11\times$
1×10^7	3410.7	3376.2	1847.4	1240.5	830.0	656.2	$5.20\times$

分析：归并排序是本实验中并行效果最好的算法， 10^7 规模下 16 线程达到 $5.2\times$ 加速比。这得益于归并排序分治结构的天然并行友好性：

1. **均衡的递归划分**：归并排序始终将数组等分为两半，子任务规模完全相同，不存在快速排序中的负载不均衡问题。
2. **任务粒度可控**：通过设置 `cutoff = 100000`，保证了每个 Task 处理足够大的数据量，有效分摊了调度开销。
3. **OMP1 与串行接近**：单线程下归并排序的 Task 版本与串行版本耗时几乎相等，说明 Task 机制对规整递归结构的额外开销极低。
4. **串行合并的限制**：加速比止步于 $5.2\times$ 而非接近 $16\times$ ，原因是归并排序的合并阶段为串行。随着线程增多，排序阶段加速而合并阶段不变，Amdahl 定律决定了最终加速比的收敛上限。

4.3.1 Amdahl 定律定量分析

根据 Amdahl 定律，当使用 p 个线程时理论加速比为：

$$S = \frac{1}{f + \frac{1-f}{p}}$$

其中 f 为串行时间比例。以归并排序 10^7 规模、16 线程数据为例，实测加速比 $S = 5.20$ ，代入公式可得：

$$5.20 = \frac{1}{f + \frac{1-f}{16}} \implies f \approx 0.13$$

即归并排序中约 13% 的时间为串行执行（主要为合并阶段的内存拷贝与归并操作），这与归并排序的结构特征一致——合并操作的时间复杂度为 $O(n)$ ，在 $n = 10^7$ 时单遍合并约 10^7 次比较/赋值操作，在总耗时中占比不可忽略。

该串行比例决定了即使线程数无限增加，归并排序的理论最大加速比上限为 $1/f \approx 7.7\times$ 。16 线程达到 $5.2\times$ 意味着在该线程数下并行效率约为 $5.2/7.7 \approx 68\%$ ，尚有提升空间。

4.3.2 理论加速与实际加速对比

表 ?? 对比了归并排序在 10^7 规模下的理论线性加速、Amdahl 理论加速（ $f = 0.13$ ）与实际加速比：

表 5: 归并排序理论加速与实际加速对比（ 10^7 规模）

线程数	线性加速	Amdahl 理论（ $f = 0.13$ ）	实际加速
1	1.00×	1.00×	1.01×
2	2.00×	1.81×	1.85×
4	4.00×	3.02×	2.75×
8	8.00×	4.58×	4.11×
16	16.00×	6.08×	5.20×

从表中可见，2 线程时实际加速已接近 Amdahl 理论值，但随着线程数进一步增加，实际加速与理论值的差距逐渐扩大。这表明除串行比例外，还受到以下因素的制约：(1) 内存带宽限制——多线程并发访问内存导致带宽竞争加剧；(2) Cache miss——多个核心同时操作不同数据区域可能引发缓存一致性开销；(3) Task 调度开销——随着任务数量增加，OpenMP 运行时的任务分配与负载均衡引入的额外时延。

4.4 三种算法综合对比

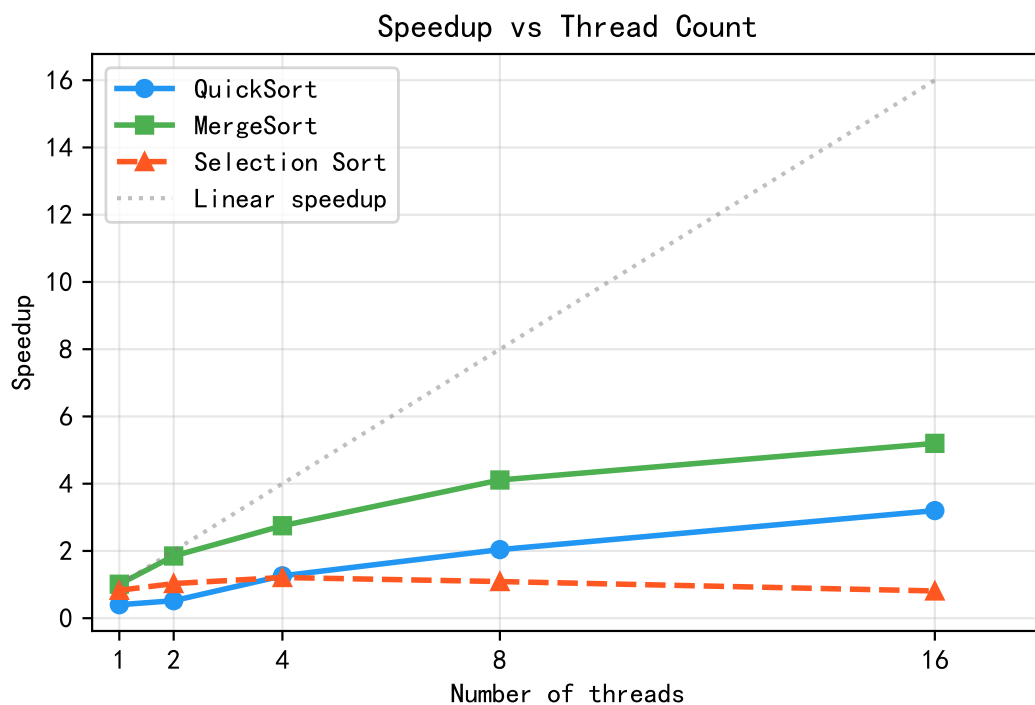


图 1: 各算法在不同线程数下的加速比对比

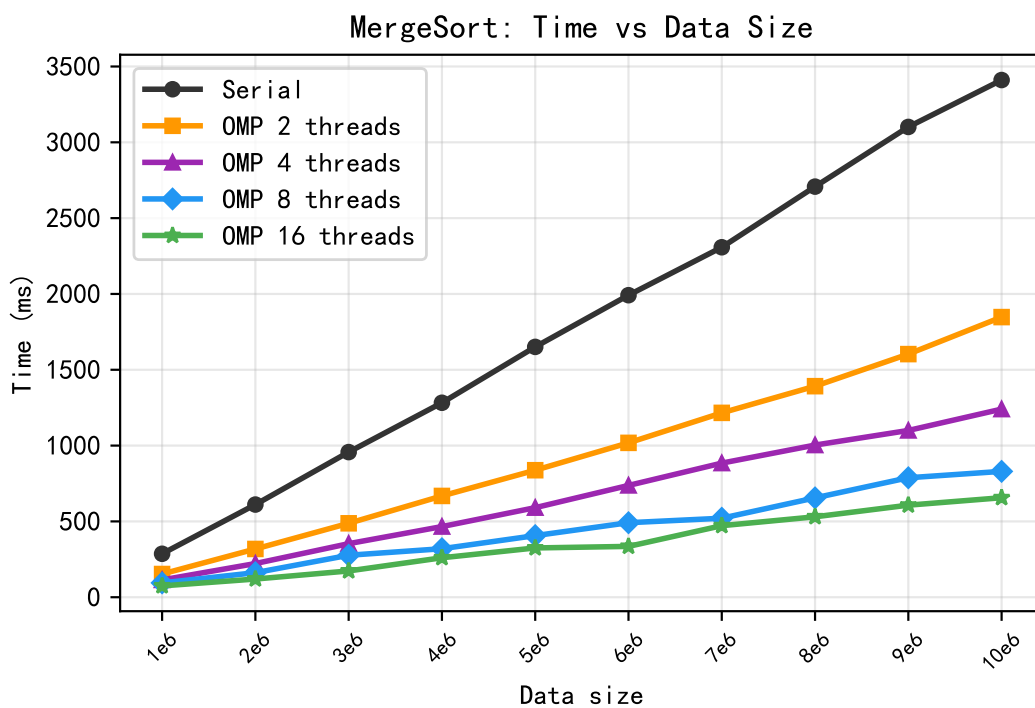


图 2: 归并排序在不同数据规模下的运行时间

表 6: 三种算法最佳加速比汇总

算法	最佳加速比	最优线程数	串行复杂度	并行瓶颈	并行适配度
选择排序	1.2×	4	$O(n^2)$	串行依赖 + 同步开销	极低
快速排序	3.3×	16	$O(n \log n)$	负载不均衡	中等
归并排序	5.5×	16	$O(n \log n)$	串行合并 + 内存拷贝	高

4.5 并行适应性分析

三种算法的并行效果差异可从算法结构角度解释：

1. **任务粒度 (Granularity)**: 归并排序的等分策略确保了每层递归两个子任务规模始终相等，调度器能充分利用所有线程。快速排序的划分依赖数据分布，极端情况下可能退化为几乎串行。
2. **串行比例 (Serial Fraction)**: 选择排序每轮迭代必须等待前一轮的最小值查找和交换完成再更新数组，串行比例接近 1。归并排序只有合并阶段是串行的，串行比例约为 $O(n)/O(n \log n) = 1/\log n$ ，随规模增大而下降，有利于并行。
3. **同步开销 (Synchronization Overhead)**: 选择排序每轮迭代有一次 `parallel/critical` 屏障同步， n 轮共 n 次同步，累计开销巨大。归并和快速排序每层递归仅需一次 `taskwait`，同步次数为 $O(\log n)$ 。

5 遇到的问题与解决

5.1 OpenMP Task 数据共享属性

初始实现未在 `#pragma omp task` 中显式指定 `firstprivate` 和 `shared` 属性，导致部分编译器版本下递归参数被默认按引用捕获，任务执行时读取的可能是已被修改的栈变量。通过在 Task 指令中显式声明 `shared(arr, temp) firstprivate(left, mid, right)` 解决了此问题。

5.2 归并排序任务爆炸

无阈值的并行归并排序在 10^7 规模下创建的递归树总节点数约为 $2^{24} - 1 \approx 1677$ 万个 Task，调度开销导致 16 线程反而比串行慢 12 倍。引入 `cutoff = 100000` 阈值后，活跃 Task 数降至可管理数量（约 $10^7/10^5 \times 2 \approx 200$ 个），并行版本首次超越串行。

6 总结

本文完成了三种经典排序算法的串行与 OpenMP 并行实现，并进行了系统的性能基准测试。主要结论如下：

1. 归并排序因规整的等分递归结构和可控的任务粒度，取得了最佳的并行加速效果（5.2× @ 16 线程），是三种算法中最适合并行化的方案。
2. 快速排序的并行效果受限于划分的负载不均衡，平均加速约 3.2×，但仍具有良好的实际应用价值。

3. 选择排序因其 $O(n^2)$ 复杂度和严格的数据依赖，并行化几乎无收益甚至负收益，这从反面验证了算法选择对并行性能的重要性。
4. OpenMP Task 机制为分治算法的并行化提供了简洁高效的编程模型——只需在递归调用前添加 `#pragma omp task`、在同步点添加 `#pragma omp taskwait` 即可实现细粒度并行，配合阈值控制可获得良好的加速效果。

通过本实验深入理解了 Amdahl 定律在并行计算中的核心地位——加速比的上限由串行比例决定，提高并行性的关键是降低串行瓶颈的占比。对归并排序的定量分析表明，其串行比例约 13%，理论最大加速上限约为 $7.7\times$ 。

参考文献

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [2] OpenMP Architecture Review Board. *OpenMP Application Programming Interface*, Version 4.5, 2015. <https://www.openmp.org/specifications/>
- [3] G. M. Amdahl. Validity of the single-processor approach to achieving large-scale computing capabilities. In *AFIPS Conference Proceedings*, 1967.
- [4] R. Chandra, L. Dagum, D. Kohr, D. Maydan, J. McDonald, R. Menon. *Parallel Programming in OpenMP*. Morgan Kaufmann, 2001.